BRUTE FORCE APPROACH

DSA Notes with Java Code, Explanation, Dry Runs & Complexity

Core idea: Brute force means trying all relevant possibilities, checking each one, and selecting the result that satisfies the problem's condition.

This guide covers five common applications:

1. Finding duplicate elements
2. Generating and checking all possible subarrays
3. Finding maximum and minimum
4. Naive string pattern matching
5. Generating all possible pairs (2-element combinations)

Language: Java **Level:** Beginner-friendly

Before You Start: Brute Force

A typical brute-force solution follows this pattern:

• **Generate:** Produce the possible candidates or positions.

• **Check:** Test each candidate against the required condition.

• **Keep / report:** Store or print the valid answer.

Important: brute force does not always mean slow. Finding the maximum and minimum in one scan takes O(n), which is optimal for an unsorted array because every element must be inspected.

Java basics used in these examples

• `arr.length` gives the number of array elements.
• Array indices start at 0 and end at `arr.length - 1`.
• `i` and `j` are loop variables; their exact roles depend on the algorithm.

1. Finding Duplicate Elements

Problem: Print every value that occurs more than once by comparing each element with the elements after it.

Example: `[10, 20, 30, 20, 10]`

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 20, 10};

        for (int i = 0; i < arr.length; i++) {
            for (int j = i + 1; j < arr.length; j++) {
                if (arr[i] == arr[j]) {
                    System.out.println("Duplicate: " + arr[i]);
                }
            }
        }
    }
}
```

Expected output:

```
Duplicate: 10
Duplicate: 20
```

How the code works

• The outer loop selects the current element at index i.

• The inner loop starts at i + 1, so the same pair is not compared twice.

• If arr[i] equals arr[j], that value has appeared at another position, so it is printed.

Dry run

i	j	Comparison
0	1	10 ≠ 20
0	2	10 ≠ 30
0	3	10 ≠ 20
0	4	10 = 10 → duplicate
1	3	20 = 20 → duplicate

Complexity: Time: $O(n^2)$ comparisons in the worst case. Auxiliary space: O(1), excluding printed output.

---

2. Generating All Possible Subarrays

Problem: Print every contiguous subarray. A subarray contains consecutive elements from the original array.

Example: `[1, 2, 3]`

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3};

        for (int i = 0; i < arr.length; i++) {
            for (int j = i; j < arr.length; j++) {
                for (int k = i; k <= j; k++) {
                    System.out.print(arr[k] + " ");
                }
                System.out.println();
            }
        }
    }
}
```

Expected output:

```
1
1 2
1 2 3
2
2 3
3
```

How the code works

• The i loop chooses the starting index.

• The j loop chooses the ending index and starts at i.

• The k loop prints every element from i through j.

Dry run

Start i	End j	Subarray
0	0	[1]
0	1	[1, 2]
0	2	[1, 2, 3]
1	1	[2]
1	2	[2, 3]
2	2	[3]

Complexity: There are n(n + 1) / 2 subarrays. Printing all their elements takes $O(n^3)$ time. Auxiliary space: O(1).

---

3. Finding Maximum and Minimum

Problem: Find the largest and smallest values by scanning the array once.

Example: `[5, 2, 9, 1, 7]`

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {5, 2, 9, 1, 7};
        int max = arr[0];
        int min = arr[0];

        for (int i = 1; i < arr.length; i++) {
            if (arr[i] > max) max = arr[i];
            if (arr[i] < min) min = arr[i];
        }

        System.out.println("Maximum: " + max);
        System.out.println("Minimum: " + min);
    }
}
```

Expected output:

```
Maximum: 9
Minimum: 1
```

How the code works

• Initialize max and min with the first element.

• Start at index 1 because index 0 is already used.

• Update max when the current element is larger; update min when it is smaller.

Dry run

Current value	Maximum so far	Minimum so far
Initial: 5	5	5
2	5	2
9	9	2
1	9	1
7	9	1

Complexity: Time: O(n). Auxiliary space: O(1). This is optimal for an arbitrary unsorted array.

---

4. Naive String Pattern Matching

Problem: Find every starting index where a pattern occurs in a larger text.

Example: `Text = "ABABCABC", Pattern = "ABC"`

```
public class Main {
    public static void main(String[] args) {
        String text = "ABABCABC";
        String pattern = "ABC";
        int n = text.length();
        int m = pattern.length();
        boolean found = false;

        for (int i = 0; i <= n - m; i++) {
            int j;
            for (j = 0; j < m; j++) {
                if (text.charAt(i + j) != pattern.charAt(j)) {
                    break;
                }
            }
            if (j == m) {
                System.out.println("Pattern found at index: " + i);
                found = true;
            }
        }
        if (!found) System.out.println("Pattern not found");
    }
}
```

Expected output:

```
Pattern found at index: 2
Pattern found at index: 5
```

How the code works

• The outer loop tries each valid starting position, from 0 through n - m.

• The inner loop compares pattern characters with the text.

• A mismatch breaks the inner loop; if j reaches m, the whole pattern matched.

Dry run

Text index	Text window	Result

0	ABA	No match
1	BAB	No match
2	ABC	Match
3	BCA	No match
4	CAB	No match
5	ABC	Match

Complexity: Worst-case time: O(n × m), where n is text length and m is pattern length. Auxiliary space: O(1).

---

5. Generating All Possible Pairs

Problem: Print every unique pair of elements, without pairing an element with itself.

Example: `[1, 2, 3, 4]`

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4};

        for (int i = 0; i < arr.length; i++) {
            for (int j = i + 1; j < arr.length; j++) {
                System.out.println("(" + arr[i] + ", " + arr[j] + ")");
            }
        }
    }
}
```

Expected output:

```
(1, 2)
(1, 3)
(1, 4)
(2, 3)
(2, 4)
(3, 4)
```

How the code works

• The outer loop chooses the first element.

• The inner loop starts at i + 1, avoiding self-pairs and reversed duplicates.

Dry run

First element	Pairs generated
1	(1, 2), (1, 3), (1, 4)
2	(2, 3), (2, 4)
3	(3, 4)
4	No pair remains

Complexity: There are n(n - 1) / 2 unique pairs. Time: $O(n^2)$. Auxiliary space: O(1).

Quick Revision: Complexity Comparison

Problem	Loop structure	Time complexity
Finding duplicates	Two nested loops	$O(n^2)$
All subarrays + print	Three nested loops	$O(n^3)$
Maximum and minimum	One loop	$O(n)$
String pattern matching	Two nested loops	$O(n \times m)$
All unique pairs	Two nested loops	$O(n^2)$

Key formulas

- Number of subarrays: $n(n + 1) / 2$
- Number of unique 2-element pairs: $n(n - 1) / 2$

Important complexity note

There are $O(n^2)$ different subarrays, but printing every element of every subarray takes $O(n^3)$ total time. Complexity depends on the work performed, not only the number of candidates.

What to remember

Brute Force = Generate possibilities + Check possibilities + Find or report the answer.

Practice questions

1. For [4, 7, 4, 9, 7], which values will the duplicate-finding code print?
2. How many subarrays does an array of 4 elements have?
3. Why does the maximum/minimum loop start at index 1?
4. What is the worst-case time complexity of naive pattern matching for text length n and pattern length m?
5. Why does pair generation start its inner loop at j = i + 1?

Self-check answers

1. 4 and 7. 2. 10 subarrays. 3. Index 0 initializes max and min. 4. $O(n \times m)$. 5. To avoid self-pairs and repeated reversed pairs.

End of notes • Practise tracing each loop on paper.